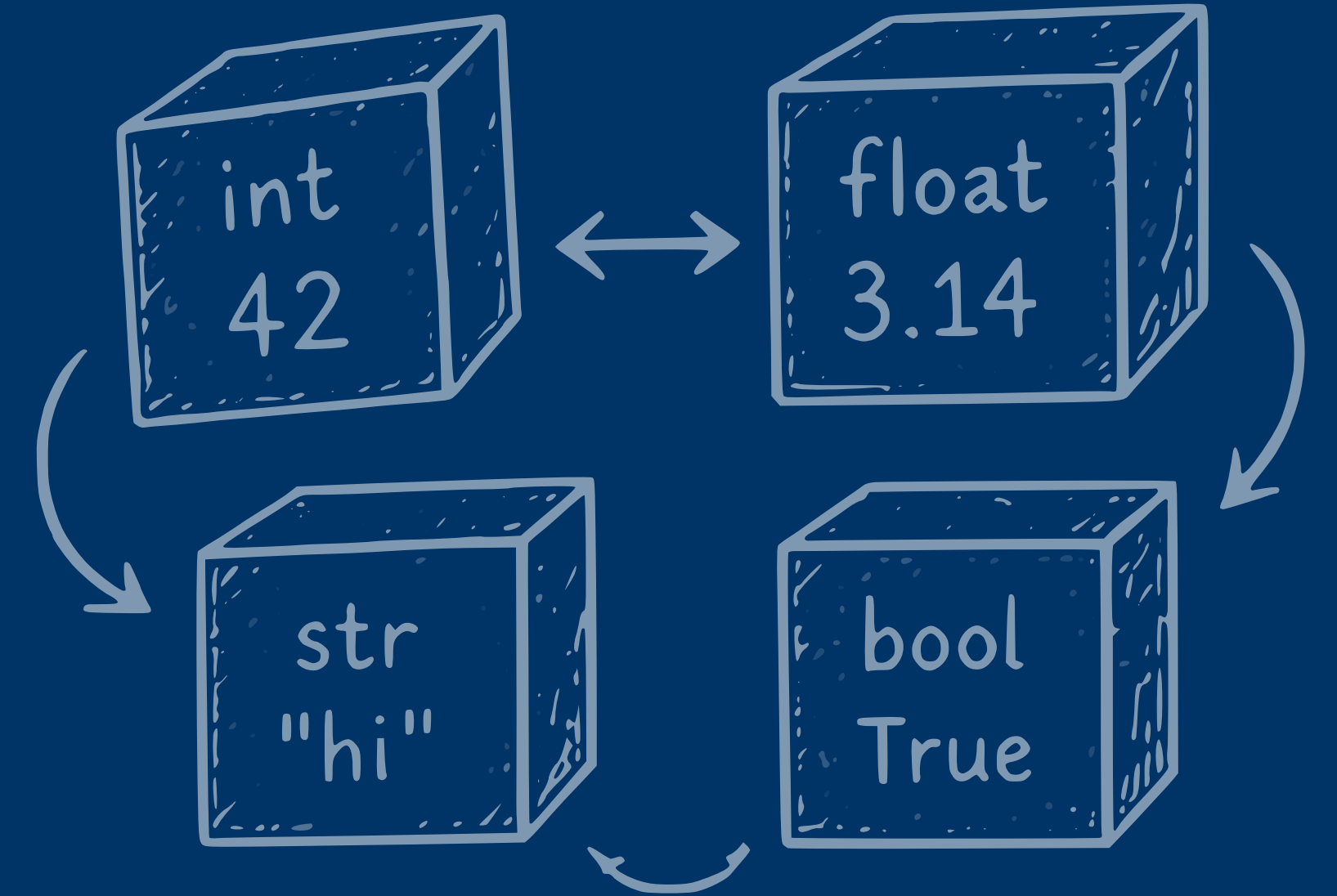


Programming

Lecture 02 — Data

Values, variables, data types and expressions



Dr. Ramon Winterhalder

Human-Centered Artificial Intelligence · A.Y. 2026/27



UNIVERSITÀ
DEGLI STUDI
DI MILANO



Where we left off

Lecture 01, in four lines

A computer does five things

store, fetch, compute, compare, repeat.

An algorithm is the idea

finite, unambiguous, with defined input and output.

Think before you type

understand, example, plan, trace — then code.

Python does what you wrote

not what you meant. Errors are how it tells you.

One thing was missing: every value we used was written into the program. Today we give values names — the step from calculator to program.

From today the plans are written in Python, not pseudocode. You can read enough of it now, and one notation is better than two.

By the end of today you can...

- give a value a name, and explain why that is not the same as equality;
- name the four types you will use for the rest of the course;
- predict both the value and the type of any expression built from them;
- say why $0.1 + 0.2$ is not 0.3 , and what that means for your programs;
- turn a short problem statement into working Python, plan first.

Every one of these is on the exam. The second half of today — predicting types — is the single most common source of lost marks.

A

Names, values and types

Assignment · int, float, str, bool · expressions

Giving a value a name

```
price = 1.50
```

price the name we choose — an identifier

= the assignment operator: "put the value on the right into the name on the left"

1.50 the value

A variable is a name attached to a value. Attach a different value later and the name follows — that is the whole idea, and it is what makes a program more than a calculation.

= is not equals

The most important slide of the day

In mathematics

$$x = x + 1$$

...has no solution. It is a false statement.

In Python

```
x = x + 1
```

...is an instruction: work out $x + 1$, then make that the new value of x .

```
x = 5          # x is 5
x = x + 1      # right side first: 5 + 1 = 6. Now x is 6
x = x * 2      # 6 * 2 = 12. Now x is 12
```

Read every assignment right to left:
work out the right-hand side, then attach the result to the name on the left.

Rules for names

Legal

price

total_2026

n

_hidden

BMI

Illegal

2nd_price

total price

class

total-price

starts with a digit

contains a space

a Python keyword

the minus sign is an operator

Letters, digits and underscores; never start with a digit; case matters, so Price and price are two different names; and you cannot reuse a word Python already owns.

Python's own words, for reference:

if else for while def class return import True False None and or not in is

Four types

Everything this term is built from these

int

whole numbers

42 -7 0

no decimal point, no size limit

float

numbers with a decimal point

3.14 -0.5 2.0

approximate — see later

str

text, in quotes

"hello" 'a' ""

single or double quotes, your choice

bool

a truth value

True False

capital T, capital F. Only two possible values

Asking Python what something is

```
>>> type(42)  
<class 'int'>
```

```
>>> type(3.14)  
<class 'float'>
```

```
>>> type("hello")  
<class 'str'>
```

```
>>> type(True)  
<class 'bool'>
```

```
>>> type(2.0)  
<class 'float'>
```

```
>>> type("42")  
<class 'str'>
```

- 2.0 is a float, not an int. The decimal point is what decides, not the value.
- "42" is a str. Quotes make it text, whatever is inside them.
- The type travels with the value, and it decides what the operators do.

Remember lecture 1: the same bits mean different things. The type is what says which.

Operators

And the order they happen in

	operator	example	result
1. brackets	()	(2 + 3) * 4	20
2. power	**	2 ** 10	1024
3. sign	-	-7	-7
4. multiply, divide	* / // %	7 // 2	3
5. add, subtract	+ -	2 + 3 * 4	14

Same order as in mathematics. When you are not sure, add brackets — they cost nothing and they document what you meant.

*Always from left to right: 8 / 4 * 2 is 4.0, not 1.0.*

Three kinds of division

<code>17 / 5</code>	<code># -> 3.4</code>	true division	always a float
<code>17 // 5</code>	<code># -> 3</code>	floor division	rounds DOWN
<code>17 % 5</code>	<code># -> 2</code>	modulo	the remainder

Why you care

- `//` and `%` together split a number into "how many fit" and "what is left".
- `%` is how you test divisibility: `n % 2 == 0` means `n` is even.
- `//` rounds down, not towards zero: `-7 // 2` is `-4`.

```
seconds = 3725
minutes = seconds // 60      # 62
leftover = seconds % 60     # 5
```

That pair — `//` and `%` — is the whole trick behind converting seconds to minutes, cents to euros, or an index to a row and a column.

Floats are approximations

The one that looks like a bug

```
>>> 0.1 + 0.2
0.30000000000000004

>>> 0.1 + 0.2 == 0.3
False
```

- A float is stored in a fixed number of bits — lecture 1, remember.
- One tenth in binary is like one third in decimal: 0.333... It never ends.
- So Python stores the closest value it can, and the tiny error shows up when you add.

Never compare two floats with `==`. Compare the difference to a small tolerance instead.

```
abs(0.1 + 0.2 - 0.3) < 1e-9      # True — this is the honest test
```

Worked example

Plan in comments, then fill it in

Step 1 — the plan

```
# the radius
# the area, from pi r squared
# the circumference
# show both, labelled
```

Step 2 — fill it in

```
PI = 3.14159
radius = 2.5

area = PI * radius ** 2
circumference = 2 * PI * radius

print("area:", area)
print("circumference:", circumference)
```

This replaces the pseudocode of lecture 1. Same discipline — decide what happens before you decide how — but in one notation, and the plan survives in the finished program as comments.

Trace it by hand

The same skill as last week, now on Python

```
a = 4
b = 10
a = a + b
b = a - b
a = a - b
```

line	a	b	what happened
a = 4	4	—	a gets 4
b = 10	4	10	b gets 10
a = a + b	14	10	4 + 10, stored in a
b = a - b	14	4	14 - 10, stored in b
a = a - b	10	4	14 - 4, stored in a

The two values have swapped, with no third variable. Work out the right-hand side first, every time, and the trick stops being a trick.

(Python has a shorter way to do this — `a, b = b, a` — which we meet in lecture 7.)

Exercises — Block A

Sheet 02, part A · 40 minutes · predict before you run

- Give the value and the type of twelve expressions.
- Trace a short sequence of assignments and report the final values.
- Decide which names are legal, and which are merely bad.
- Fix four expressions whose brackets are in the wrong place.
- Work out what `//` and `%` give for a negative number, and explain why.

Write the answer down before you type. Every question here can be answered on paper, and on the exam it will have to be.



B

Writing programs people can read

Conversion · shorthand · naming · the errors you will meet

Changing type on purpose

<code>int("42")</code>	<code># -></code>	<code>42</code>	text to whole number
<code>float("3.14")</code>	<code># -></code>	<code>3.14</code>	text to decimal
<code>str(42)</code>	<code># -></code>	<code>"42"</code>	number to text
<code>int(3.9)</code>	<code># -></code>	<code>3</code>	float to int – CHOPS, never rounds
<code>float(3)</code>	<code># -></code>	<code>3.0</code>	int to float

- `int()` on a float throws the decimals away. `int(3.9)` is 3 and `int(-3.9)` is -3.
- To round properly, use `round(3.9)`, which gives 4.

You will need `int()` constantly from next week, because everything the user types arrives as text — even when it looks like a number.

When conversion fails

```
>>> int("3.14")
ValueError: invalid literal for int() with base 10: '3.14'
```

- The type was right — it was a str, as int() expects. The contents were wrong.
- That is what ValueError means: right kind of thing, unusable value.
- Compare with TypeError from last week: wrong kind of thing entirely.

error	means	example
NameError	there is no such name	print(x) before x exists
TypeError	wrong kind of value	"a" + 1
ValueError	right kind, bad contents	int("3.14")

Shorthand

The same thing, written shorter

shorthand	means	
total += 5	total = total + 5	used constantly from lecture 4
total -= 5	total = total - 5	
total *= 2	total = total * 2	
count += 1	count = count + 1	counting things

There is no new idea here — it is the same assignment, written with fewer characters. Read it right to left like any other.

Careful: += is one token. There is no such thing as =+, and writing it gives you plain assignment with a positive sign.

Naming things well

The cheapest way to make code readable

Works, but poor

```
x = 68
y = 1.75
z = x / y ** 2
print(z)
```

Better

```
mass_kg = 68
height_m = 1.75
bmi = mass_kg / height_m ** 2
print("BMI:", bmi)
```

- Lower case, words joined by underscores: mass_kg, not MassKg or masskg.
- Say what it is, not what type it is: price, not price_float.
- Short names are fine where the meaning is obvious and local — i, n, x.
- CONSTANTS in capitals, by convention: PI, MAX_ATTEMPTS.

Worked example

Seconds to hours, minutes and seconds

The plan

```
# how many seconds in total  
# whole hours, and what is left  
# whole minutes from the rest  
# the seconds that remain  
# print it, labelled
```

```
total = 7385  
  
hours = total // 3600  
rest = total % 3600  
  
minutes = rest // 60  
seconds = rest % 60  
  
print(hours, "h", minutes, "m", seconds, "s")
```

Output: 2 h 3 m 5 s. The whole program is // and % applied twice — the pattern from Block A, used for something real.

Exercises — Block B

Sheet 02, part B · 40 minutes · plan in comments first

Write

- A circle: area and circumference.
- A shopping total with VAT added.
- Seconds to h:m:s, from your own plan.
- Swap two variables without a third.

Read and predict

- Five programs: what does each print?
- Three broken ones: name the error.
- One that runs but gives the wrong answer — find out why.

Start every "write" question with the comment plan. If you cannot write the comments, you do not yet understand the problem — and no amount of typing will fix that.

Today in one slide

A variable is a name attached to a value

and = attaches it. Right-hand side first, always.

Four types

int, float, str, bool — and the type decides what the operators do.

Floats are approximate

$0.1 + 0.2$ is not 0.3. Never compare them with ==.

// and % split a number

how many fit, and what is left over.

Convert on purpose

int(), float(), str() — and int() chops rather than rounds.

Plan in comments

then fill the code in underneath. The plan stays as documentation.

Before next time

- Finish sheet 02. Solutions go online later.
- If Python is still not running on your own machine, come and find me this week — from here on you will want it.
- Read chapter 1.2 of Sedgewick, Wayne & Dondero for a second voice on types and expressions.
- Bring the laptop.

Next: Lecture 03 — input, output, and making decisions. Your programs start reacting to what they are given.